I. COMPUTER HARDWARE

A. Definition

Hardware refers to the **physical and tangible components** of a computer system that you can see and touch.

B. Major Categories of Hardware

1. Input Devices Used to enter data and instructions into the computer.

- Keyboard
- Mouse
- Scanner
- Microphone
- Webcam

2. Processing Device

Responsible for processing data.

Central Processing Unit (CPU)

- Known as the **brain of the computer**
- Performs calculations and controls operations

Main Parts of CPU:

- **ALU (Arithmetic Logic Unit)** – Performs arithmetic and logical operations
- **Control Unit (CU)** – Directs and coordinates computer activities

3. Output Devices Used to display processed information.

- Monitor
- Printer
- Speaker
- Projector

4. Storage Devices Used to store data and programs.

a. Primary Storage (Main Memory)

- **RAM (Random Access Memory)**
  - Volatile
  - Temporary storage
- **ROM (Read Only Memory)**
  - Non-volatile
  - Permanent storage

b. Secondary Storage

- Hard Disk Drive (HDD)
- Solid State Drive (SSD)
- USB Flash Drive
- CD/DVD

C. Hardware Examples Summary

Category	Examples
Input	Keyboard, Mouse
Output	Monitor, Printer
Processing	CPU
Storage	HDD, SSD

II. COMPUTER SOFTWARE

A. Definition

Software refers to a **set of programs and instructions** that tell the computer what to do.

B. Types of Software

1. System Software Controls and manages computer hardware and system operations.

Examples:

- Operating Systems:
  - Windows
  - Linux
  - macOS
- Device Drivers
- Utility Programs

Functions of System Software:

- Manages memory
- Controls hardware
- Runs application software

2. Application Software Designed to perform **specific tasks for users.**

Examples:

- Microsoft Word
- Excel
- PowerPoint
- Google Chrome
- Photoshop

Categories:

- General-purpose software
- Specialized software

3. Programming Software Used to create other software.

Examples:

- Compilers
- Interpreters
- IDEs (Visual Studio Code, NetBeans)

III. HARDWARE VS SOFTWARE

Hardware	Software
Physical components	Programs and instructions
Can be touched	Cannot be touched
Requires software to operate	Requires hardware to run
Example: Keyboard	Example: Windows OS

BINARY NUMBER SYSTEM

I. INTRODUCTION TO NUMBER SYSTEMS

A. Definition of Number System

A **number system** is a way of representing numbers using a set of symbols and rules.

Common Number Systems:

- **Decimal** – Base 10 (0–9)
- **Binary** – Base 2 (0 and 1)
- **Octal** – Base 8 (0–7)
- **Hexadecimal** – Base 16 (0–9, A–F)

II. BINARY NUMBER SYSTEM

A. Definition

The **Binary Number System** is a **base-2 number system** that uses only **two digits: 0 and 1**.

Why Computers Use Binary

- Electronic circuits have **two states**: ON (1) and OFF (0)
- Binary is **reliable, simple, and efficient** for computers

III. BASIC BINARY TERMS

1. Bit

- Short for **binary digit**
- Smallest unit of data
- Can be **0 or 1**

2. Byte

- Group of **8 bits**
- Represents one character (e.g., letter, number)

3. MSB and LSB

- **MSB (Most Significant Bit)** – Leftmost bit
- **LSB (Least Significant Bit)** – Rightmost bit

I. INTRODUCTION TO DATABASE

A. Definition

A **database** is an **organized collection of related data** stored electronically so it can be **easily accessed, managed, and updated**.

II. DATABASE VS FILE SYSTEM

File System

- Stores data in separate files
- High data redundancy
- Low security

Database System

- Centralized data storage
- Reduced redundancy
- Improved security and integrity

III. DATABASE MANAGEMENT SYSTEM (DBMS)

A. Definition

A **DBMS** is software that allows users to **create, store, manage, and retrieve data** in a database.

Examples of DBMS

- MySQL
- Oracle
- PostgreSQL
- SQL Server
- MongoDB

Functions of DBMS

- Data storage and retrieval
- Data security
- Backup and recovery
- Data integrity

IV. DATABASE COMPONENTS

1. Table Collection of related data organized in rows and columns

2. Record (Row) A complete set of related data in a table

3. Field (Column) A single piece of data in a record

V. KEYS IN DATABASE

1. Primary Key

- Uniquely identifies each record
- Cannot be null or duplicated

2. Foreign Key

- Links one table to another
- Refers to the primary key of another table

3. Candidate Key Possible primary keys

4. Composite Key Combination of two or more fields as a primary key

VI. TYPES OF DATABASES

1. Relational Database

- Data stored in tables
- Uses SQL
- Example: MySQL

2. Non-Relational Database (NoSQL)

- Data stored as documents, key-value, etc.
- Example: MongoDB

3. Centralized Databas Stored in one location

4. Distributed Database Stored in multiple locations

VII. STRUCTURED QUERY LANGUAGE (SQL)

A. Definition

SQL is a standard language used to **communicate with relational databases**.

Common SQL Commands

- **SELECT** – Retrieve data
- **INSERT** – Add new data
- **UPDATE** – Modify data
- **DELETE** – Remove data
- **CREATE** – Create tables

VIII. DATA INTEGRITY AND SECURITY

Data Integrity Ensures data is **accurate, consistent, and reliable.**

Types:

- Entity integrity
- Referential integrity

Data Security

Protects data from:

- Unauthorized access
- Data loss
- Data breaches

Methods:

- User authentication
- Access control
- Backup

IX. DATABASE NORMALIZATION

A. Definition

Normalization is the process of **organizing data** to reduce redundancy and improve integrity.

Normal Forms (Basic):

- **1NF** – No repeating groups
- **2NF** – Fully dependent on primary key
- **3NF** – No transitive dependency

X. IMPORTANCE OF DATABASE IN IT

- Efficient data management
- Supports decision-making

- Improves data security
- Used in websites, applications, and systems

I. INTRODUCTION TO PROGRAMMING

A. Definition of Programming

Programming is the process of writing instructions (code) that a computer follows to perform tasks.

B. Programming Languages

A **programming language** is a formal language used to communicate instructions to a computer.

II. COMMON PROGRAMMING CONCEPTS

1. Variables

- Used to store data
- Value can change during execution

2. Data Types

Type	Description	Example
Integer	Whole numbers	10
Float	Decimal numbers	3.14
Character	Single symbol	'A'
String	Text	"Hello"
Boolean	True/False	true

3. Operators

a. Arithmetic Operators

+ - * / %

b. Relational Operators

== != > < >= <=

c. Logical Operators

&& || !

4. Control Structures

a. Conditional Statements

if

if-else

switch

b. Looping Statements

for

while

do-while

5. Functions

- A block of code that performs a specific task
- Improves reusability and readability

III. JAVASCRIPT REVIEW

A. Definition

JavaScript is a **client-side scripting language** mainly used to make web pages **interactive**.

B. Key Features

- Runs in the browser
- Event-driven
- Used for form validation and UI interaction

C. Basic Concepts

- Variables: var, let, const
- Output: console.log()
- Events: onclick, onchange

IV. C PROGRAMMING REVIEW

A. Definition

C is a **procedural programming language** used for **system and application programming**.

B. Key Features

- Fast execution
- Uses functions
- Close to hardware

C. Basic Concepts

- Header files: #include <stdio.h>
- Main function
- Input/Output: printf(), scanf()

V. PHP REVIEW

A. Definition

PHP (Hypertext Preprocessor) is a **server-side scripting language** used for **web development**.

B. Key Features

- Executes on the server
- Can interact with databases
- Embedded in HTML

C. Basic Concepts

- Variables start with $
- Output using echo
- Used with MySQL

VI. COMPARISON OF LANGUAGES

Feature	JavaScript	C	PHP
Type	Scripting	Procedural	Scripting
Runs On	Browser	Compiler	Server
Main Use	Web interaction	System programs	Web backend
Database Access	No (direct)	No	Yes

VII. COMMON PROGRAMMING ERRORS

1. **Syntax Error** Incorrect language rules

2. **Logical Error** Program runs but gives wrong output

3. **Runtime Error** Error during execution

VIII. IMPORTANCE OF CODING IN IT

- Automates tasks
- Builds applications and systems
- Solves real-world problems
- Core skill for IT professionals

Arrays

Definition of Array:

An array is a special variable used to store multiple values in a single variable, organized in an indexed list.

Arrays start at index **0**.

OBJECT-ORIENTED PROGRAMMING (OOP)

I. INTRODUCTION TO OOP

A. Definition

Object-Oriented Programming (OOP) is a **programming paradigm** that organizes software design around **objects** rather than actions, focusing on **data (attributes)** and **behavior (methods)**.

B. Advantages of OOP

- **Modularity** – Code is organized into classes and objects
- **Reusability** – Classes can be reused in different programs
- **Maintainability** – Easy to update and modify
- **Encapsulation** – Hides internal details
- **Polymorphism** – Same interface, different implementation

II. CORE CONCEPTS OF OOP

1. Class

- A **blueprint or template** for creating objects
- Defines **attributes (properties)** and **methods (functions)**

2. Object

- A **real instance** of a class
- Has specific values for attributes and can perform methods

3. Encapsulation

- **Hides internal data** of a class from outside access
- Access is through **methods (getters/setters)**
- Protects integrity of data

4. Abstraction

- **Shows only essential features** to the user
- **Hides unnecessary implementation details**

5. Inheritance

- **Allows one class to acquire properties/methods of another**
- Promotes **code reusability**

6. Polymorphism

- **Ability to take many forms**
- Same method can behave differently based on object or context

Types:

- **Compile-time (Overloading)** – Same method name, different parameters
- **Run-time (Overriding)** – Child class redefines parent method

III. OOP TERMINOLOGY

Term	Description
Class	Blueprint for objects
Object	Instance of a class
Attribute	Data stored in a class
Method	Function defined in a class
Constructor	Special method to initialize objects
Destructor	Special method to clean up objects

IV. OOP IN PROGRAMMING LANGUAGES

- **Java** – Fully OOP
- **C++** – Supports OOP
- **Python** – OOP supported, flexible
- **PHP** – Supports classes and objects

I. INTRODUCTION TO SQL

A. Definition

SQL (Structured Query Language) is a **standard language used to communicate with relational databases** to **store, manipulate, and retrieve data**.

B. SQL Command Categories

Category	Purpose
DDL (Data Definition Language)	Define or modify database structures (tables, indexes)
DML (Data Manipulation Language)	Manipulate data (insert, update, delete)
Query	Retrieve data from tables (SELECT)

II. DDL (DATA DEFINITION LANGUAGE)

1. **CREATE** Used to **create databases or tables**
2. **ALTER** Used to **modify an existing table**
3. **DROP** Used to **delete a table or database**

III. DML (DATA MANIPULATION LANGUAGE)

1. **INSERT** Add new data to a table
2. **UPDATE** Modify existing data
3. **DELETE** Remove data from a table

IV. QUERY COMMANDS

1. **SELECT** Retrieve data from tables
2. **WHERE Clause** Filter records based on conditions
3. **ORDER BY** Sort results ascending (ASC) or descending (DESC)
4. **GROUP BY** Group rows that share a column value
5. **JOIN** Combine rows from two or more tables

Sorting Algorithm A step-by-step procedure used to arrange elements of a list or array in a specific order (ascending or descending).

Ascending Order Arranging data from smallest to largest (e.g., 1–10, A–Z).

Descending Order Arranging data from largest to smallest (e.g., 10–1, Z–A).

Comparison-Based Sorting Sorting method that determines order by comparing two elements (e.g., Bubble Sort, Quick Sort, Merge Sort).

Non-Comparison Sorting Sorting method that does not compare elements directly but uses counting or indexing (e.g., Counting Sort).

Time Complexity A measure of how the running time of an algorithm grows relative to input size (expressed using Big-O notation).

Space Complexity The amount of memory an algorithm uses relative to input size.

Big-O Notation A mathematical notation used to describe the upper bound of an algorithm's performance (e.g., O(n), $O(n^2)$, O(n log n)).

Stable Sorting A sorting algorithm where elements with equal values maintain their original order after sorting.

Unstable Sorting A sorting algorithm where equal elements may change their original order.

In-Place Sorting A sorting algorithm that requires only a constant amount of extra memory (O(1)).

Divide and Conquer An algorithmic strategy that divides a problem into smaller subproblems, solves them recursively, and then combines the results.

TYPES OF SORTING ALGORITHMS (TERMS)

Bubble Sort A simple sorting algorithm that repeatedly compares and swaps adjacent elements if they are in the wrong order.

Selection Sort A sorting algorithm that repeatedly selects the smallest (or largest) element and places it in its correct position.

Insertion Sort A sorting algorithm that builds the sorted list one element at a time by inserting elements into their proper position.

Merge Sort A divide-and-conquer sorting algorithm that splits the array into halves, sorts each half, and merges them back together.

Quick Sort A divide-and-conquer sorting algorithm that selects a pivot element and partitions the array into smaller and larger elements.

Heap Sort A sorting algorithm that uses a binary heap data structure to sort elements efficiently.

Counting Sort A non-comparison sorting algorithm that counts occurrences of each value and calculates positions.

Pivot (Quick Sort Term) The element chosen to partition the array into smaller and larger elements in Quick Sort.

Partitioning The process of rearranging elements around a pivot in Quick Sort.

Recursion A process where a function calls itself to solve smaller instances of the same problem (used in Merge Sort and Quick Sort).

Worst Case The maximum time an algorithm may take for the worst possible input.

Best Case The minimum time an algorithm may take for the most favorable input.

Average Case The expected running time of an algorithm for random input.

PERFORMANCE TERMS

$O(n^2)$ – Quadratic Time Performance grows proportionally to the square of input size (slow for large data).

O(n log n) – Log-Linear Time Efficient performance for large datasets.

O(n) – Linear Time Performance grows directly proportional to input size.